7.4 Using curl to simulate CORS requests

When building a new CORS implementation, it's often useful to craft HTTP requests to the server and observe the results. One way to do this is to create a web page with an `XMLHttpRequest` that performs exactly the request you need. While this works, it can grow tedious to edit an HTML page, refresh it in the browser, then check the results in the developer tools. Curl is a command-line tool that lets you craft HTTP requests to the server and view the results. Let's look at how you can use curl to make requests to the sample server.

7.4.1 Making CORS requests using curl

Curl comes preinstalled on most Linux and Mac systems, but you'll have to download it for Windows (appendix B goes into detail on how to install curl). You can see if you have curl installed on your computer by typing `curl --version`. If this returns an error, you'll need to install curl from http://curl.haxx.se/.

> **NOTE** Some versions of Window's Powershell already have an alias to `curl` that points to the `Invoke-Webrequest` cmdlet. If typing `curl --version` returns an error about "Bad Gateway," you're using the wrong curl. You can work around this issue by installing and using the curl tool from http://curl.haxx.se/.

Curl can be used to send HTTP requests to servers. Let's start with a simple example. The following command can be used to send an HTTP request to Google:

```
curl http://www.google.com
```

If you type in this command, it will return the HTML response from Google.com. Next, let's turn this curl request on the sample server. The following command hits the sample blogging API:

```
curl http://127.0.0.1:9999/api/posts
```

If you run this command, you'll receive the response `Forbidden`. While the request was sent to the server, this message doesn't shed any insight into what is wrong, especially because there aren't HTTP headers in the response. Curl's `--verbose` option can be used to display the request and response headers. To view the HTTP headers, add the `--verbose` option to the curl request:

```
curl --verbose http://127.0.0.1:9999/api/posts
```

This command will not only print `Forbidden`, but also the request and response headers, as shown in figure 7.13.

The request headers are prefixed with `>`, while the response headers are prefixed with `<`. Looking at the response headers, you can clearly see that the Access-Control-Allow-Origin header is missing.

If you look closely at the request headers, you can also see that the Origin header is missing. The browser automatically adds the Origin header. When using curl, you need to manually add the Origin header. You can set headers on curl requests by